Plantilla del alumnado · CRUD Academia

Este proyecto es la **base de examen/práctica** para trabajar una aplicación Java por capas con Maven.

La plantilla ya deja **todo definido**:

- paquetes
- interfaces
- clases
- firmas de métodos
- `Main` completo
- tests de servicio

Tu trabajo consiste en **terminar la implementación** de las funcionalidades nuevas sin romper lo que ya funciona de `Alumno`.

Qué ya funciona

Está completamente implementado el bloque de `Alumno`:

- modelo `Alumno`
- repositorio `AlumnoRepositorioCSV`
- servicio `AlumnoService`
- validaciones básicas
- parte correspondiente del `Main`

Puedes usar ese bloque como referencia para mantener el mismo estilo en el resto de capas.

Qué debes implementar

1. Gestión completa de `Asignatura`

Debes completar:

- `modelo/Asignatura.java`
- `persistencia/IAsignaturaRepositorio.java`
- `persistencia/impl/AsignaturaRepositorioCSV.java`
- `servicio/IAsignaturaService.java`
- `servicio/AsignaturaService.java`
- validaciones asociadas en `Validaciones.java`

Requisitos de `Asignatura`

- atributos: `codigo`, `nombre`, `horasSemanales`
- constructores
- getters y setters
- `equals()` y `hashCode()` por `codigo`

- `toString()`
- `toCsv()`

Operaciones mínimas

- crear asignatura
- listar asignaturas
- buscar asignatura por código
- actualizar asignatura
- eliminar asignatura

Los datos deben persistirse en:

```
src/main/resources/asignaturas.csv
```

---

2. Gestión completa de `AlumnoMatriculado`

Debes completar:

- `modelo/AlumnoMatriculado.java`
- `persistencia/IAlumnoMatriculadoRepositorio.java`
- `persistencia/impl/AlumnoMatriculadoRepositorioCSV.java`
- `servicio/IAlumnoMatriculadoService.java`
- `servicio/AlumnoMatriculadoService.java`

Requisitos de `AlumnoMatriculado`

- debe **extender** a `Alumno`
- debe añadir `List<Asignatura> asignaturas`
- debe incluir constructores
- getters y setters
- método para añadir asignaturas sin duplicados
- método para eliminar asignaturas
- `toString()`
- `toCsv()`

Lógica de negocio obligatoria

- un alumno no puede tener la misma asignatura dos veces
- obtener el **número de asignaturas** como `int`
- comprobar si está matriculado en una asignatura con `boolean`

Operaciones mínimas

- crear alumno matriculado
- listar alumnos matriculados
- buscar alumno matriculado

- actualizar alumno matriculado
- eliminar alumno matriculado
- matricular en asignatura
- desmatricular de asignatura
- consultar número de asignaturas
- comprobar si está matriculado o no

Los datos deben persistirse en:

```
src/main/resources/alumnos_matriculados.csv
```

Formato orientativo:

```
AM001|Lucia|19|1DAM|PROG,BBDD
AM002|Mario|20|1DAM|LM
```

---

3. Main

La clase `Main` ya está preparada con el menú completo.

Cuando termines las capas nuevas, las opciones del menú deberán funcionar:

- crear asignatura
- listar asignaturas
- buscar asignatura
- modificar asignatura
- eliminar asignatura
- crear alumno matriculado
- listar alumnos matriculados
- matricular en asignatura
- desmatricular de asignatura
- consultar número de asignaturas
- consultar matrícula en una asignatura
- buscar alumno matriculado

Estructura del proyecto

```
src/
├─ main/
│  ├─ java/com/docente/
│  │  ├─ app/
│  │  │  └─ Main.java
│  │  ├─ ficheros/
│  │  ├─ modelo/
```

```
│   │   │   ├─ Alumno.java
│   │   │   ├─ AlumnoMatriculado.java
│   │   │   ├─ Asignatura.java
│   │   │   └─ PersonaAbstracta.java
│   │   ├─ persistencia/
│   │   │   ├─ IAlumnoRepositorio.java
│   │   │   ├─ IAlumnoMatriculadoRepositorio.java
│   │   │   ├─ IAsignaturaRepositorio.java
│   │   │   └─ impl/
│   │   │       ├─ AlumnoRepositorioCSV.java
│   │   │       ├─ AlumnoMatriculadoRepositorioCSV.java
│   │   │       └─ AsignaturaRepositorioCSV.java
│   │   ├─ servicio/
│   │   │   ├─ AlumnoService.java
│   │   │   ├─ AlumnoMatriculadoService.java
│   │   │   ├─ AsignaturaService.java
│   │   │   ├─ IAlumnoService.java
│   │   │   ├─ IAlumnoMatriculadoService.java
│   │   │   └─ IAsignaturaService.java
│   │   ├─ utils/
│   │   └─ validacion/
│   └─ resources/
│       ├─ alumnos.csv
│       ├─ alumnos_matriculados.csv
│       └─ asignaturas.csv
└─ test/
    ├─ java/
    │   └─ com/docente/
    │       ├─ modelo/
    │       └─ servicio/
    └─ resources/
        ├─ alumnos.csv
        ├─ alumnos_matriculados.csv
        └─ asignaturas.csv
```

Ejecución del proyecto

Requisitos

- Java 17
- Maven 3.9 o superior

Compilar

```
mvn clean compile
```

Ejecutar tests

```
mvn clean test
```

Importante sobre los CSV de tests

Cada vez que ejecutes `mvn test`, el `pom.xml` está configurado para copiar los ficheros `*.csv` de:

```
src/test/resources
```

a:

```
src/main/resources
```

Esto permite que los tests partan siempre de un estado conocido.

Ejecutar la aplicación

```
mvn exec:java
```

También puedes ejecutar directamente la clase:

```
com.docente.app.Main
```

desde tu IDE.

Sobre los tests

Se incluyen tests de ejemplo a nivel de servicio:

- `AlumnoServiceTest`
- `AsignaturaServiceTest`
- `AlumnoMatriculadoServiceTest`

Qué debes esperar al principio

- los tests de `Alumno` deben servirte como referencia
- los tests de `Asignatura` y `AlumnoMatriculado` **pueden fallar** hasta que completes la implementación

Tu objetivo es que, al final, todo el proyecto compile y todos los tests pasen.

Sugerencia de trabajo

Sigue este orden:

1. completa `Asignatura` modelo
2. completa repositorio CSV de `Asignatura`
3. completa `AsignaturaService`
4. completa `AlumnoMatriculado` modelo
5. completa repositorio CSV de `AlumnoMatriculado`
6. completa `AlumnoMatriculadoService`
7. revisa validaciones
8. prueba el menú en `Main`
9. ejecuta `mvn test`

Pistas

- usa `Alumno` y `AlumnoService` como patrón
- en `equals()` y `hashCode()` de `Asignatura`, trabaja con `codigo`
- para evitar duplicados de asignaturas, puedes usar `contains()` si `equals()` está bien hecho
- al guardar `AlumnoMatriculado`, solo necesitas persistir los **códigos** de las asignaturas en el CSV